Logging

Sommario

Sommario....1
1. Introduzione....1
2. Tipologie di log....2
2.1. Log comuni....2
2.1.1. Pattern....2
2.1.2. Tracing....3
2.1.3. Livelli di log....3
2.2. Log di performance....4
2.2.1. Pattern....4
2.2.2. Log di performance infrastrutturali....5
2.2.2.1. Log di perfomance per le API....5
2.2.2.2. Log di performance per i servizi REST integrati....5
2.3. Log body di risposta delle chiamate REST in errore....6
3. Implementazione....7
3.1. Configurabilità offerta dalla libreria....7
4. DevOps....9
4.1. Configurazione....9
4.1.1. Livelli log....9
4.1.1.1. Ispezione e settaggio tramite console JMX....11
4. Ispezione log....12
4.1. LogAnalytics....12
4.2 Kubectl/OpenLens....12

1. Introduzione

Lo scopo del seguente documento è quello di descrivere le tipologie, le modalità di produzione dei log applicativi e la loro configurabilità.

2. Tipologie di log

2.1. Log comuni

Rappresentano le più comuni informazioni prodotte durante l'esecuzione del codice applicativo e a loro volta sono classificabili in:

- log tecnici
  - Informazioni principalmente prodotte dalle librerie di terze parti utilizzate e riportanti dati contestuali all'operazione demandatagli.
  - Mirano ad informare su cosa, a basso livello, starebbe operando l'applicativo, piuttosto che le motivazioni che lo hanno portato ad eseguirlo.
- log di processo
  - Informazioni prodotte durante l'esecuzione delle logiche di business e volutamente inserite all'interno del codice.
  - Hanno lo scopo di informare dell'esecuzione e dello stato di avanzamento dei processi, riportando dati contestuali volti a identificare il processo attivato, i suoi parametri ed eventuali risultati ottenuti, anche intermedi.

2.1.1. Pattern

I log di questa tipologia soddisfano il seguente pattern:

yyyy-MM-dd HH:mm:ss.SSS [appName] [userId] [traceId] level [thread] [logger] - Messaggio

Volto ad enfatizzare il timestamp di esecuzione, dove:

- appName: nome dell'applicativo in oggetto
- userId: identificativo tecnico dell'utente (non è un PII)

  (mappedExternalUserId, V. Architettura microservizi | 3.1. Classi di utenze
-)
- traceId: identificativo che correla i log, anche inter-microservizi, associati allo stesso contesto di elaborazione
- level: il livello di log
- thread: gli ultimi 15 caratteri del nome del thread

- logger: il nome del logger, di default il nome della classe, con le sole iniziali dei package a cui appartiene
- MESSAGGIO: il corpo del log

Es:

2024-12-03 10:39:05.029 [p4pa-auth]
[_3uEBnSDUgDt4U7EDvbPb4L-VuKv0x4YhvAjzAsPDzk=]
[a2d0035216ed7f93feb82b0451caebe0] INFO [nio-8080-exec-9]
[g.p.p.a.s.e.ValidateExternalTokenService] - SubjectToken authorized

2.1.2. Tracing

I log sono corredati di un traceId generato all'avvio di un processo/richiesta ed inviato durante le richieste REST interne, in modo da poterle relazionare ed individuare con semplicità.

2.1.3. Livelli di log

In ordine dal più grave a quello di mera informazione quasi trascurabile:

1. ERROR: Indicano un evento imprevisto, causato da un fattore interno all'applicativo, che potrebbe compromettere l'esito del processo in esecuzione.
    - Eventuali fattori esterni, come input imprevisti/errati da parte dell'utente, come ad esempio un identificativo non esistente, non rientrano in questa categoria, ma piuttosto andrebbero catalogati come info.
2. warn: Indicano una situazione anomala, generata da fattori interni all'applicativo, ma non bloccante perché opzionale e/o per via dell'attuazione di meccanismi di ripristino/fallback.
3. INFO: Descrivono il verificarsi degli eventi principali, riportando dati contestuali utili a riconoscere cosa sta avvenendo.
    - In caso di logiche di business, descrivono le principali fasi di cui si compongono i flussi, riportando ad esempio come dati contestuali i parametri dei metodi.
4. DEBUG: Descrivono in maniera più dettagliata e verbosa gli eventi verificatisi, offrendo anche una granularità più fine sulle particolari operazioni eseguite.

5. trace: Rappresenta la modalità più verbosa e scrupolosa, atta a tracciare la più minima informazione inerente al processo in esecuzione, non necessariamente di primario interesse.

2.2. Log di performance

Rappresentano particolari log di processo volti ad informare circa le tempistiche di esecuzione del flusso in oggetto.

2.2.1. Pattern

Aderiscono allo stesso pattern dei log comuni di processo, utilizzando come logger il prefisso PERFORMANCE_LOG. seguito da un tag identificativo del flusso in esecuzione .

Es:

2025-02-17 10:36:02.582 [p4pa-auth] [EUqKiD1psLrGNuLxCGzriy-royPlBvuyeJMc0dxaxNs=] [904b8a748e44bb1f8139e0cf64735d8f] INFO [at-handler-7567] [PERFORMANCE_LOG.API_REQUEST] - [POST /payhub/auth/token] Time occurred to perform business logic: 125 ms. HttpStatus: 200

Il loro livello è configurabile in base al tempo registrato. Di default vengono rispettati i seguenti livelli:

- ERROR: Tempistica registrata > 300s
- warn: Tempistica registrata > 60s
- INFO: Altrimenti

L'uso del campo logger consente di poter applicare un filtro sui livelli di log diverso per ogni processo.

Es:

```
logging:

 level:

   PERFORMANCE_LOG: "\${LOG_LEVEL_PERFORMANCE_LOG:INFO}"
```

PERFORMANCE_LOG.API_REQUEST: "\${LOG_LEVEL_PERFORMANCE_LOG_API_REQUEST:\${logging.level.PERFORMANCE_LOG}}"

PERFORMANCE_LOG.REST_INVOKE: "\${LOG_LEVEL_PERFORMANCE_LOG_REST_INVOKE:\${logging.level.PERFORMANCE_LOG}}"

2.2.2. Log di performance infrastrutturali

Esistono alcuni log di performance la cui registrazione è configurata di default in ogni applicativo.

2.2.2.1. Log di perfomance per le API

Registrate mediante un Filtro applicato a tutte le richieste servite dalla componente DispatcherServlet e configurate con il logger PERFORMANCE_LOG.API_REQUEST.

Oltre alla tempistica, riportano:

- Come prefisso metodo HTTP e path della richiesta
- Come suffisso lo stato HTTP restituito

Es:

2025-02-17 10:36:02.582 [p4pa-auth] [EUqKiD1psLrGNuLxCGzriy-royPlBvuyeJMc0dxaxNs=] [904b8a748e44bb1f8139e0cf64735d8f] INFO [at-handler-7567] [PERFORMANCE_LOG.API_REQUEST] - [POST /payhub/auth/token] Time occurred to perform business logic: 125 ms. HttpStatus: 200

2.2.2.2. Log di performance per i servizi REST integrati

I servizi REST integrati mediante la componente di Spring RestTemplate vengono corredati di un Interceptor atto a registrare e loggare il tempo di esecuzione utilizzando come logger PERFORMANCE_LOG.REST_INVOKE.

Oltre alla tempistica, riportano:

- Come prefisso metodo HTTP e URL del servizio invocato
- Come suffisso lo stato HTTP restituito

Es:

2025-02-17 10:37:04.541 [p4pa-auth] [] [8145ed306765cfcdc22473b46c69c14f] INFO [at-handler-7581] [PERFORMANCE_LOG.REST_INVOKE] - [GET http://p4pa-organization-microservice-chart:8080/crud/organizations/search/findByIpaCode?ipaCode=DEMO] Time occurred to perform business logic: 23 ms. HttpStatus: 200

2.3. Log body di risposta delle chiamate REST in errore

Viene fornita l'implementazione di un ResponseErrorHandler da configurare opzionalmente nei RestTemplate utilizzati nelle integrazioni REST al fine di loggare il body e lo stato HTTP restituito se inatteso.

Non si tratta di una funzionalità automaticamente configurata, in quanto occorre essere certi che i body di errore non possano contenere PII.

Loggherà con livello INFO e riporterà:

- Metodo HTTP e URL invocata
- Stato e messaggio HTTP restituito
- L'eccezione sollevata da RestTemplate
- Il body della risposta

Es:

2025-02-17 11:42:59.550 [p4pa-auth] [] [bc381eefcb10a6ca05c2d88fb7f1aac7] INFO [at-handler-8555] [REST_INVOKE.ORGANIZATION] - GET http://p4pa-organization-microservice-chart:8080/crud/organizations/search/findByIpaCode?ipaCode= Returned status 404 NOT_FOUND and resulted on exception NotFound - 404 Not Found on GET request for "http://p4pa-organization-microservice-chart:8080/crud/organizations/search/findByIpaCode": "{"code":"ORGANIZATION_NOT_FOUND","message":"EntityRepresentationModel not found"}":

{"code":"ORGANIZATION_NOT_FOUND","message":"EntityRepresentationModel not found"}

3. Implementazione

Gli applicativi realizzati fanno uso della libreria Simple Logging Facade for Java (SLF4J), ossia una libreria che si limita a fornire un approccio standardizzato alla produzione dei log mediante l'uso di interfacce in maniera del tutto libera dalla particolare implementazione scelta.

Tale libreria consente:

- La definizione di un nome da associare alla particolare istanza del logger ottenuto.
    - Solitamente si usa definire una costante per classe ottenuta creando un logger avente come nome la Class stessa (nel particolare si è fatto uso della libreria Lombok la quale mette a disposizione una annotazione di classe, *@Slf4j*, che si occupa di definire la costante all'interno della classe annotata)
- La definizione, durante la produzione dei log stessi, del livello di appartenenza.
- L'uso di pattern all'interno dei log, con placeholder ({}) che verranno risolti solamente nel caso in cui il log supera il filtro impostato.

Nel particolare, si è optato come implementazione per lo standard suggerito dal framework Spring stesso, ossia Logback.

Quest'ultimo consente la configurazione di vari *appender* volti ad individuare la destinazione dei log (ad esempio su console, su file, su coda kafka, su server syslog, ...) ed il pattern da rispettare nella costruzione della stringa finale.

3.1. Configurabilità offerta dalla libreria

Slf4j consente la definizione di un filtro associando un livello minimo di log ai nomi dei logger utilizzati.

Qualora si fosse fornito un Class come nome, sarà possibile configurarne i livelli di log sia in maniera specifica sul nome della classe, sia su prefissi del package a cui appartiene, in modo tale da fornire lo stesso livello di log a porzioni significative dell'applicativo rispettandone il design di costruzione.

Spring si integra con la libreria Slf4j accettandone la configurazione mediante file di properties/yml al cui interno è possibile far riferimento a variabili d'ambiente, permettendone quindi una configurazione semplificata.

La definizione dei livelli di log è demandata al particolare applicativo, per cui si rimanda ad una ispezione del particolare applicativo di interesse. Una sua possibile configurazione è questa:

```
logging:

 level:

   root: "\${LOG_LEVEL_ROOT:INFO}"

   it.gov.pagopa: "\${LOG_LEVEL_PAGOPA:INFO}"

   org.springframework: "\${LOG_LEVEL_SPRING:INFO}"

   org.springframework.boot.availability:
"\${LOG_LEVEL_SPRING_BOOT_AVAILABILITY:DEBUG}"

   it.gov.pagopa.payhub.auth.exception.AuthExceptionHandler:
"\${LOGGING_LEVEL_API_REQUEST_EXCEPTION:INFO}"

   org.springdoc.core.utils.SpringDocAnnotationsUtils:
"\${LOG_LEVEL_SPRING_DOC:ERROR}"

   PERFORMANCE_LOG: "\${LOG_LEVEL_PERFORMANCE_LOG:INFO}"

   PERFORMANCE_LOG.API_REQUEST:
"\${LOG_LEVEL_PERFORMANCE_LOG_API_REQUEST:\${logging.level.PERFORMANC
E_LOG}}"
```

PERFORMANCE_LOG.REST_INVOKE: "\${LOG_LEVEL_PERFORMANCE_LOG_REST_INVOKE:\${logging.level.PERFORMANCE_LOG}}"

4. DevOps

4.1. Configurazione

4.1.1. Livelli log

Ad eccezione dei log di audit, è solitamente possibile configurare il livello minimo dei log utilizzando le variabili d'ambiente.

Si rimanda ad una ispezione del particolare applicativo al fine di osservare le particolari variabili a disposizione. Le più comuni sono:

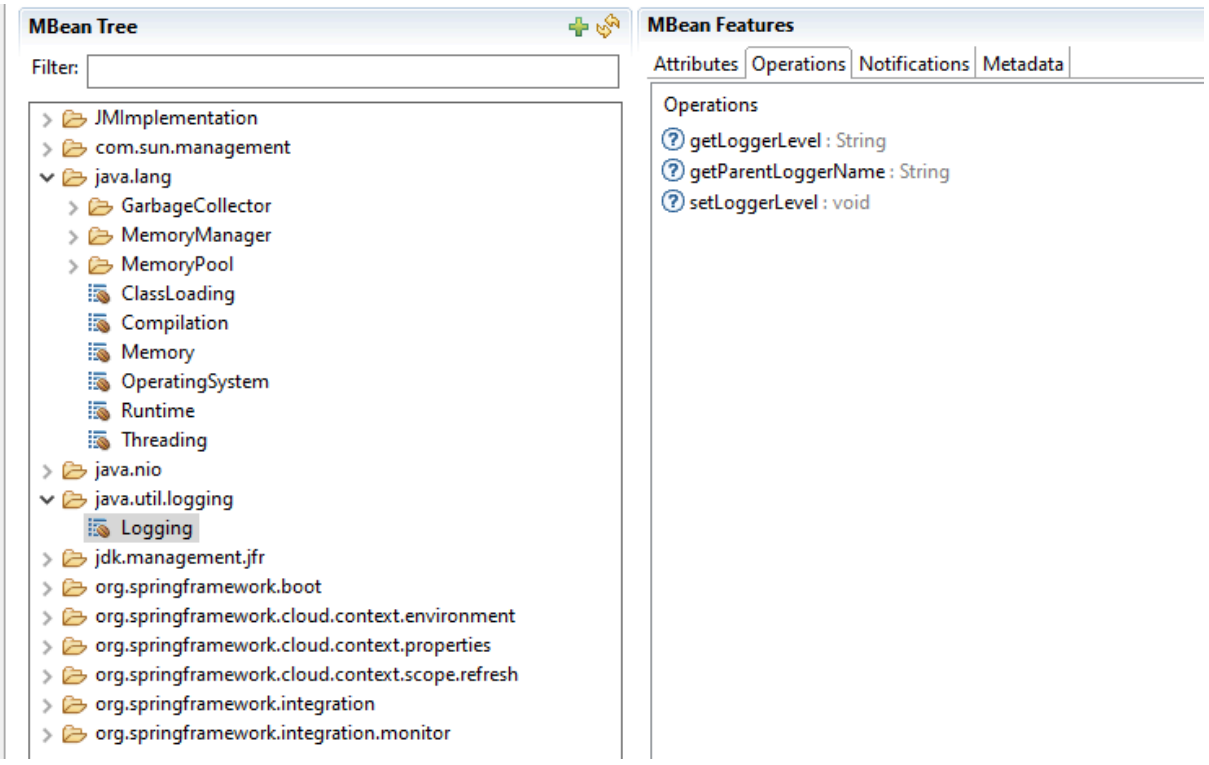
ENV	Descrizione	Default
LOG_LEVEL_ROOT	Livello base applicato in assenza di una definizione più specifica.	INFO
LOG_LEVEL_PAGOPA	Livello base applicato, in assenza di una definizione più specifica, alle classi custom.	INFO
LOG_LEVEL_SPRING	Livello applicato alla libreria Spring.	INFO

LOG_LEVEL_SPRING_BOOT_AVAILABILITY	Consente la visualizzazione degli eventi di cambio della availability dell'applicativo.	DEBUG
LOGGING_LEVEL_API_REQUEST_EXCEPTION	Livello applicato al gestore delle eccezioni lanciate e risalite fino ai Controller delle API REST esposte.	INFO
LOG_LEVEL_SPRING_DOC	Livello applicato alla libreria SpringDoc (la libreria usata per la generazione automatica dell'openApi e per l'esposizione della pagina di Swagger)	ERROR
LOG_LEVEL_IO_SWAGGER	Livello applicato alla libreria Swagger	warn
LOG_LEVEL_PERFORMANCE_LOG	Livello base applicato per i log di performance	INFO

LOG_LEVEL_PERFORMANCE_LOG_API_REQUEST	Livello applicato per i log di performance associati alle API esposte	INFO
LOG_LEVEL_PERFORMANCE_LOG_REST_INVOKE	Livello applicato per i log di performance associati alle API REST invocate	INFO

4.1.1.1. Ispezione e settaggio tramite console JMX

È comunque possibile ispezionare e modificare a runtime, senza la necessità di riavviare l'applicativo, i livelli di log mediante la console JMX qualora sia attiva e raggiungibile nell'ambiente di interesse.



4. Ispezione log

4.1. LogAnalytics

Il cluster AKS utilizzato è configurato per redirigere i log prodotti sulla console dagli applicativi ivi deployati all'interno di LogAnalytics ed in particolare all'interno della tabella ContainerLog.

V. PU - Runbook ispezione Log

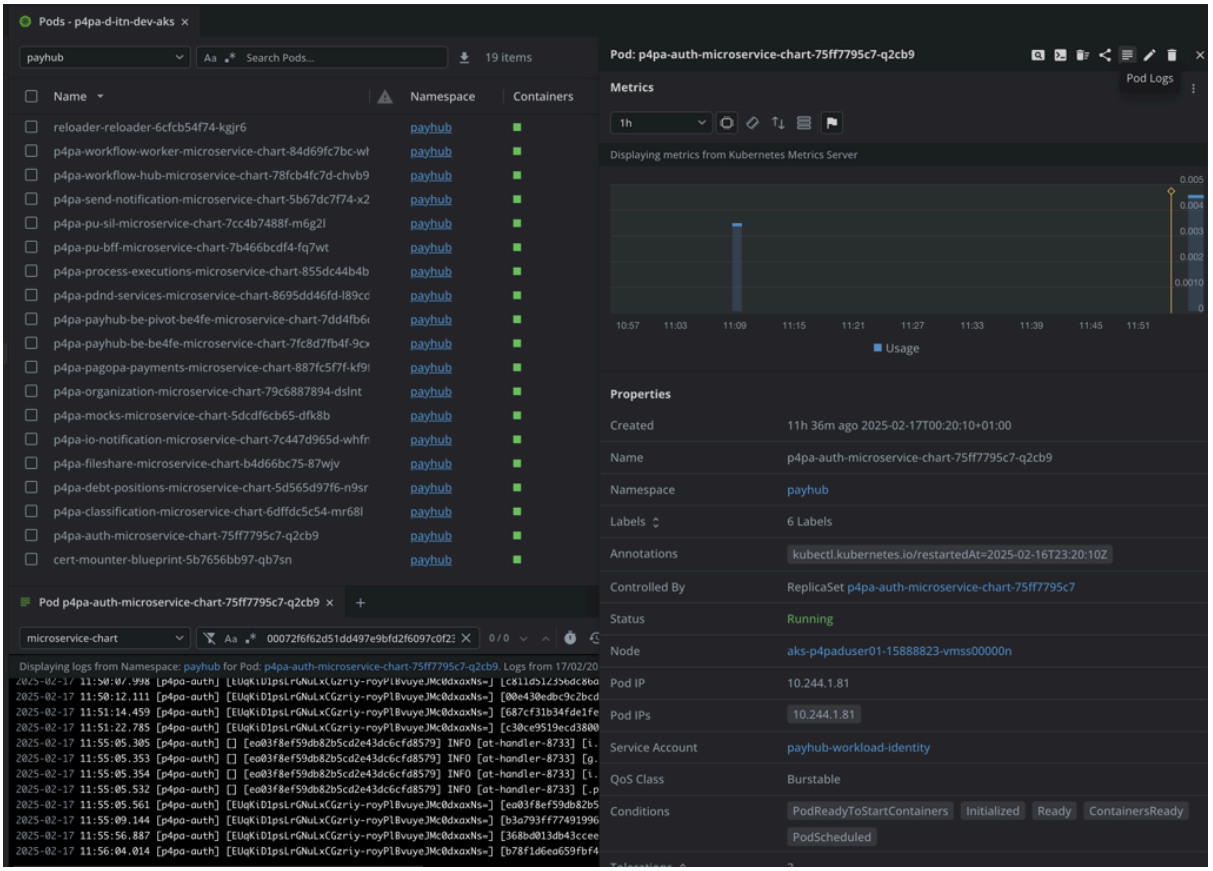
e PU - Runbook query log per utente/traceId

per esempi su come ispezionare i log usando LogAnalytics.

4.2 Kubectl/OpenLens

Esistono varie utilities sfruttabili al fine di osservare i log prodotti dai singoli pod di K8s, come ad esempio kubectl e OpenLens.

```
antonio.torre@LAP-MBP16-N02-0520 ~ % kubectl get pods -n payhub
NAME                                                      READY   STATUS    RESTARTS   AGE
cert-mounter-blueprint-5b7656bb97-qb7sn                   1/1     Running   0          3d9h
p4pa-auth-microservice-chart-75ff7795c7-q2cb9             1/1     Running   0          11h
p4pa-classification-microservice-chart-6dffdc5c54-mr68l   1/1     Running   0          68m
p4pa-debt-positions-microservice-chart-5d565d97f6-n9srd   1/1     Running   0          69m
p4pa-fileshare-microservice-chart-b4d66bc75-87wjv         1/1     Running   0          55m
p4pa-io-notification-microservice-chart-7c447d965d-whfn9  1/1     Running   0          55m
p4pa-mocks-microservice-chart-5dcdf6cb65-dfk8b            1/1     Running   0          3d9h
p4pa-organization-microservice-chart-79c6887894-dslnt     1/1     Running   0          55m
p4pa-pagopa-payments-microservice-chart-887fc5f7f-kf9fr   1/1     Running   0          55m
p4pa-payhub-be-be4fe-microservice-chart-7fc8d7fb4f-9cxs6  1/1     Running   0          3d9h
p4pa-payhub-be-pivot-be4fe-microservice-chart-7dd4fb6db6-hg472  1/1  Running  0        8d
p4pa-pdnd-services-microservice-chart-8695dd46fd-l89cd    1/1     Running   0          68m
p4pa-process-executions-microservice-chart-855dc44b4b-mlj2r  1/1  Running  0          68m
p4pa-pu-bff-microservice-chart-7b466bcdf4-fq7wt           1/1     Running   0          55m
p4pa-pu-sil-microservice-chart-7cc4b7488f-m6g2l           1/1     Running   0          126m
p4pa-send-notification-microservice-chart-5b67dc7f74-x2rdt  1/1  Running  0          68m
p4pa-workflow-hub-microservice-chart-78fcb4fc7d-chvb9     1/1     Running   0          81m
p4pa-workflow-worker-microservice-chart-84d69fc7bc-whnzv  1/1     Running   0          55m
reloader-reloader-6cfcb54f74-kgjr6                        1/1     Running   0          3d9h
antonio.torre@LAP-MBP16-N02-0520 ~ % kubectl logs -f -n payhub p4pa-auth-microservice-chart-75ff7795c7-q2cb9
```


Pods - p4pa-d-itn-dev-aks
payhub
Search Pods...
19 items
Name
Namespace
Containers
reloader-reloader-6cfcb54f74-kgjr6
p4pa-workflow-worker-microservice-chart-84d69fc7bc-wl
p4pa-workflow-hub-microservice-chart-78fcb4fc7d-chvb9
p4pa-send-notification-microservice-chart-5b67dc7f74-x2
p4pa-pu-sil-microservice-chart-7cc4b7488f-m6g2l
p4pa-pu-bff-microservice-chart-7b466bcdf4-fq7wt
p4pa-process-executions-microservice-chart-855dc44b4b
p4pa-pdnd-services-microservice-chart-8695dd46fd-l89cc
p4pa-payhub-be-pivot-be4fe-microservice-chart-7dd4fb6
p4pa-payhub-be-be4fe-microservice-chart-7fc8d7fb4f-9cx
p4pa-pagopa-payments-microservice-chart-887fc5f7f-kf9
p4pa-organization-microservice-chart-79c6887894-dslnt
p4pa-mocks-microservice-chart-5dcdf6cb65-dfk8b
p4pa-io-notification-microservice-chart-7c447d965d-whfr
p4pa-fileshare-microservice-chart-b4d66bc75-87wjv
p4pa-debt-positions-microservice-chart-5d565d97f6-n9sr
p4pa-classification-microservice-chart-6dffdc5c54-mr68l
p4pa-auth-microservice-chart-75ff7795c7-q2cb9
cert-mounter-blueprint-5b7656bb97-qb7sn
payhub
Pod p4pa-auth-microservice-chart-75ff7795c7-q2cb9
microservice-chart
Displaying logs from Namespace: payhub for Pod: p4pa-auth-microservice-chart-75ff7795c7-q2cb9. Logs from 17/02/20
Pod: p4pa-auth-microservice-chart-75ff7795c7-q2cb9
Pod Logs
Metrics
1h
Displaying metrics from Kubernetes Metrics Server
Usage
Properties
Created
11h 36m ago 2025-02-17T00:20:10+01:00
Name
p4pa-auth-microservice-chart-75ff7795c7-q2cb9
Namespace
payhub
Labels
6 Labels
Annotations
kubectl.kubernetes.io/restartedAt=2025-02-16T23:20:10Z
Controlled By
ReplicaSet p4pa-auth-microservice-chart-75ff7795c7
Status
Running
Node
aks-p4paduser01-15888823-vmss00000n
Pod IP
10.244.1.81
Pod IPs
10.244.1.81
Service Account
payhub-workload-identity
QoS Class
Burstable
Conditions
PodReadyToStartContainers
Initialized
Ready
ContainersReady
PodScheduled